

SAT Solver — Brute Force

Project Report

Name: Mohamed Ramy Farouk

ID: 320240014

1. Project Title

SAT Solver — Brute Force

2. Project Idea

The SAT Solver project is a web application designed to solve propositional logic formulas using the brute-force method.

The system allows the user to enter logical expressions using logical operators such as AND, OR, NOT, implication, and biconditional.

The project checks all possible truth assignments for the variables in the formula until it finds a satisfying assignment that makes the formula true.

If no assignment satisfies the formula, the system reports that the formula is unsatisfiable.

The project was built using Python Flask for the backend and HTML, CSS, and JavaScript for the frontend.

3. Project Description

This project is a complete SAT Solver web application with a modern graphical interface. The application evaluates propositional logic formulas using a brute-force SAT solving engine.

The user can enter logical formulas, solve formulas using brute-force enumeration, view truth tables, export CSV files, and access built-in examples of classical rules of inference.

The system evaluates every possible combination of truth values for the variables and determines whether the formula can be satisfied.

4. Features of the Project

- Modern glassmorphism user interface
- Animated aurora background
- Formula input with logical operator buttons
- ASCII shortcut support
- Brute-force SAT solving engine
- Automatic truth table generation
- CSV export functionality

- Built-in examples for classical logic rules
- Local history storage for solved formulas

5. Technologies Used

Backend:

- Python
- Flask

Frontend:

- HTML
- CSS
- JavaScript

6. Explanation of the Brute-Force Method

The brute-force method is a direct and simple approach used to solve SAT problems by checking every possible truth assignment for the variables in a logical formula.

For example, if a formula contains the variables A, B, and C, the solver generates all possible combinations of TRUE and FALSE values for these variables.

The number of possible combinations depends on the number of variables.

For n variables, the solver generates 2^n possible truth assignments.

The system evaluates the formula for every generated assignment until all possible cases are tested.

A formula is called SATISFIABLE if there is at least one truth assignment that makes the formula evaluate to TRUE.

Example:

$A \vee B$

If:

$A = \text{TRUE}$

$B = \text{FALSE}$

Then the formula becomes TRUE, so the formula is satisfiable.

A formula is called UNSATISFIABLE if no possible truth assignment can make the formula TRUE.

Example:

$A \wedge \neg A$

This formula can never become TRUE because if A is TRUE then $\neg A$ becomes FALSE, and if A

is FALSE then A itself becomes FALSE.

The brute-force approach guarantees accurate results because every possible truth assignment is checked systematically and completely.

7. How the Project Works

1. The user enters a propositional logic formula.
2. The parser analyzes and validates the formula syntax.
3. The solver extracts all variables from the formula.
4. The brute-force engine generates all possible truth assignments.
5. Each assignment is evaluated against the formula.
6. If a satisfying assignment exists, it is displayed to the user.
7. A complete truth table is generated automatically.

8. How to Run the Project

Step 1: Install Requirements

```
pip install -r requirements.txt
```

Step 2: Run the Flask Application

```
python app.py
```

Step 3: Open the Browser

```
http://localhost:5000
```

9. Project Structure

SAT-SOLVER/

```
|
├── app.py
├── requirements.txt
├── backend/
│   ├── __init__.py
│   ├── parser.py
│   ├── solver.py
│   └── utils.py
├── frontend/
│   ├── templates/
│   │   └── index.html
│   ├── static/
│   │   ├── css/
│   │   │   └── style.css
│   │   └── js/
│   │       └── scripts.js
```

10. File Description

`app.py`

The main Flask application file. It starts the server, handles routes, receives formulas from the frontend, and sends results back to the user interface.

`requirements.txt`

Contains all required Python libraries and dependencies needed to run the project.

`backend/__init__.py`

Initializes the backend package and allows Python to recognize the backend folder as a module.

`backend/parser.py`

Responsible for parsing logical formulas entered by the user. It validates syntax and converts the formula into a structure that can be evaluated by the solver.

`backend/solver.py`

Contains the brute-force SAT solving engine. It generates every possible truth assignment and checks whether the formula evaluates to true.

`backend/utils.py`

Contains helper and utility functions used throughout the backend logic.

`frontend/templates/index.html`

The main HTML page of the application. It contains the user interface structure, formula input field, buttons, and result sections.

`frontend/static/css/style.css`

Responsible for the visual design and styling of the application including glassmorphism effects, layout styling, animations, colors, and responsive design.

`frontend/static/js/scripts.js`

Handles frontend interactivity using JavaScript including sending formulas to the backend, updating results dynamically, managing operator buttons, and handling local history and CSV export.

11. Output of the Project

The application outputs:

- Whether the formula is satisfiable or not
- A satisfying assignment if one exists
- A complete truth table for all variable combinations
- CSV export of the truth table

12. Conclusion

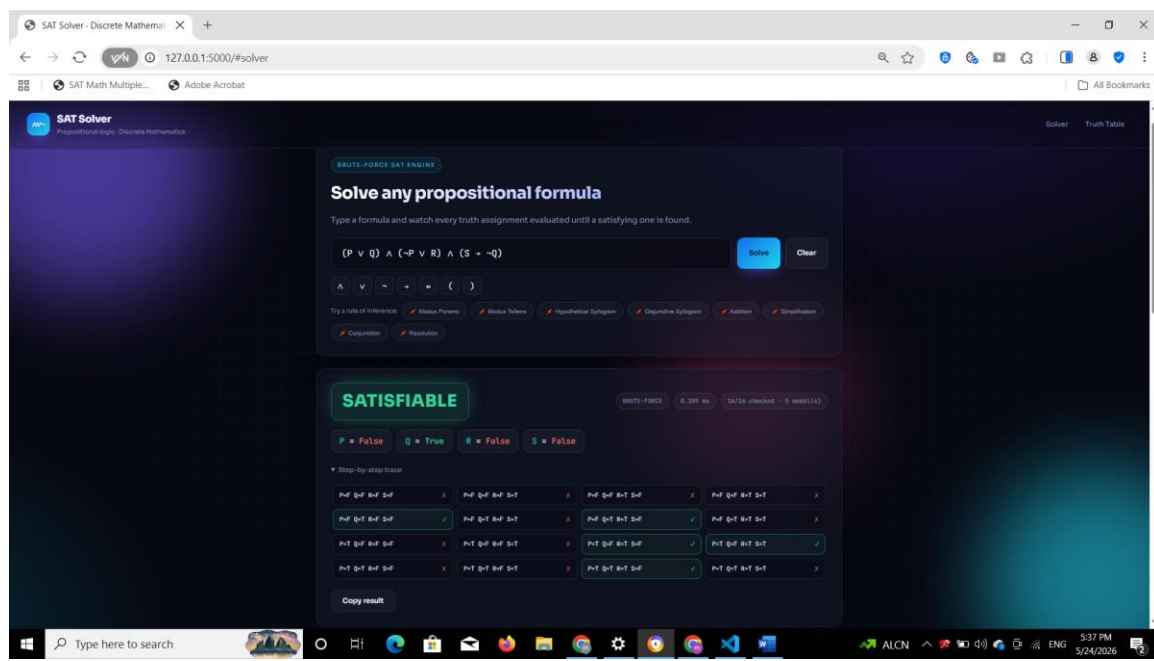
The SAT Solver project demonstrates how propositional logic formulas can be solved using a brute-force approach.

The project combines logical reasoning with web development technologies to provide an interactive and user-friendly SAT solving system.

The project also demonstrates the integration between frontend and backend technologies using Flask and JavaScript.

13. Screenshots of the Output

Satisfiable Equation:



Truth table

P	Q	R	S	P
F	F	F	F	F
F	F	F	T	F
F	F	T	F	F
F	F	T	T	F
F	T	F	F	T
F	T	F	T	F
F	T	T	F	T
F	T	T	T	F
T	F	F	F	F
T	F	F	T	F
T	F	T	F	T
T	F	T	T	F
T	T	F	F	T
T	T	F	T	F
T	T	T	F	T
T	T	T	T	F

Report CSV

SAT Solver
Propositional Logic - Discrete Mathematics

Solver Truth Table

T	T	T	F	T
T	T	T	T	F

History

$(P \vee Q) \wedge (\neg P \vee R) \wedge (S \wedge \neg Q)$
100%

$((P \vee Q) \wedge \neg P) \rightarrow Q$
100%

$(P \wedge Q) \wedge (Q \wedge R) \wedge (R \wedge S) \wedge (S \wedge P) \wedge (P \vee Q \vee R \vee S) \wedge (\neg P \vee \neg Q \vee \neg R \vee \neg S)$
10000%

$(P \wedge Q) \wedge (Q \wedge R) \wedge (R \wedge S) \wedge (S \wedge \neg P) \wedge (P \vee S) \wedge (\neg Q \vee \neg R)$
100%

$((P \wedge Q) \wedge \neg Q) \rightarrow \neg P$
100%

$(P \wedge Q) \wedge P \wedge \neg Q$
10000%

$((P \wedge Q) \wedge (Q \wedge R)) \rightarrow (P \wedge R)$
100%

$P \wedge \neg P$
10000%

$((P \vee Q) \wedge (\neg P \vee R)) \rightarrow (Q \vee R)$
100%

$(P \wedge Q) \rightarrow (P \wedge Q)$
100%

$(P \wedge Q) \rightarrow P$
100%

$P \rightarrow (P \vee Q)$
100%

$((P \wedge Q) \wedge P) \rightarrow Q$
100%

Unsatisfiable Equation:

The screenshot shows a web browser window with the URL `127.0.0.1:5000/#solver`. The page is titled "SAT Solver - Discrete Mathematics" and features a dark theme. The main heading is "Solve any propositional formula". Below it, a text input field contains the formula $(P \wedge Q) \wedge (R \wedge \neg P) \wedge (S \wedge \neg Q) \wedge (R \vee S)$. To the right of the input are "Solve" and "Clear" buttons. Below the input field, a row of inference rules is displayed: **Try a rule of inference:** Modus Ponens Modus Tollens Horizontal Syntegm Disjunctive Syntegm Addition Simplification Componetion Resolution. The "Solve" button is highlighted in blue. Below the inference rules, a large red box with the word "UNSATISFIABLE" in white text is centered. To the right of this box, a status bar shows "BRUTE-FORCE", "0.004 ms", "18/18 checked", and "0 model(s)". Below the "UNSATISFIABLE" box, a message states: "No assignment satisfies the formula. The formula is a contradiction over its variables." Underneath this message, a "Step-by-step trace" section shows a table of truth assignments for variables P, Q, R, S, and their combinations, with each row marked with a red 'X' indicating that no assignment satisfies the formula. A "Copy result" button is located at the bottom of the trace section. The browser's taskbar at the bottom shows various application icons and the system clock indicating 5:42 PM on 5/24/2026.

SAT Solver - Discrete Mathematics

BRUTE-FORCE SAT ENGINE

Solve any propositional formula

Type a formula and watch every truth assignment evaluated until a satisfying one is found.

$(P \wedge Q) \wedge (R \wedge \neg P) \wedge (S \wedge \neg Q) \wedge (R \vee S)$ Solve Clear

Try a rule of inference: Modus Ponens Modus Tollens Horizontal Syntegm Disjunctive Syntegm Addition Simplification Componetion Resolution

UNSATISFIABLE

BRUTE-FORCE 0.004 ms 18/18 checked 0 model(s)

No assignment satisfies the formula. The formula is a contradiction over its variables.

Step-by-step trace

P=F Q=F R=F S=F	X	P=F Q=F R=F S=T	X	P=F Q=F R=T S=F	X	P=F Q=F R=T S=T	X
P=F Q=T R=F S=F	X	P=F Q=T R=F S=T	X	P=F Q=T R=T S=F	X	P=F Q=T R=T S=T	X
P=T Q=F R=F S=F	X	P=T Q=F R=F S=T	X	P=T Q=F R=T S=F	X	P=T Q=F R=T S=T	X
P=T Q=T R=F S=F	X	P=T Q=T R=F S=T	X	P=T Q=T R=T S=F	X	P=T Q=T R=T S=T	X

Copy result

SAT Solver - Discrete Mathematics

127.0.0.1:5000/#solver

SAT Math Multiple... Adobe Acrobat

SAT Solver

Copy result

Truth Table

P	Q	R	S	T
F	F	F	F	F
F	F	F	F	T
F	F	F	T	F
F	F	F	T	T
F	F	T	F	F
F	F	T	F	T
F	F	T	T	F
F	F	T	T	T
F	T	F	F	F
F	T	F	F	T
F	T	F	T	F
F	T	F	T	T
F	T	T	F	F
F	T	T	F	T
F	T	T	T	F
F	T	T	T	T
T	F	F	F	F
T	F	F	F	T
T	F	F	T	F
T	F	F	T	T
T	F	T	F	F
T	F	T	F	T
T	F	T	T	F
T	F	T	T	T
T	T	F	F	F
T	T	F	F	T
T	T	F	T	F
T	T	F	T	T
T	T	T	F	F
T	T	T	F	T
T	T	T	T	F
T	T	T	T	T

Report CSV

Windows taskbar: Type here to search, 5:42 PM 5/24/2026

SAT Solver - Discrete Mathematics

127.0.0.1:5000/#solver

SAT Math Multiple... Adobe Acrobat

SAT Solver

P	Q	R	S	T
T	T	T	F	F
T	T	T	T	F

History

- $(P \wedge Q) \wedge (R \wedge \neg P) \wedge (S \wedge \neg Q) \wedge (R \vee S)$ (SAT)
- $(P \vee Q) \wedge (\neg P \vee R) \wedge (S \wedge \neg Q)$ (SAT)
- $((P \vee Q) \wedge \neg P) \rightarrow Q$ (SAT)
- $(P \wedge Q) \wedge (Q \wedge R) \wedge (R \wedge S) \wedge (S \wedge P) \wedge (P \vee Q \vee R \vee S) \wedge (\neg P \vee \neg Q \vee \neg R \vee \neg S)$ (SAT)
- $(P \wedge Q) \wedge (Q \wedge R) \wedge (R \wedge S) \wedge (S \wedge \neg P) \wedge (P \vee S) \wedge (\neg Q \vee \neg R)$ (SAT)
- $((P \wedge Q) \wedge \neg Q) \rightarrow \neg P$ (SAT)
- $(P \wedge Q) \wedge P \wedge \neg Q$ (SAT)
- $((P \wedge Q) \wedge (Q \wedge R)) \rightarrow (P \wedge R)$ (SAT)
- $P \wedge \neg P$ (SAT)
- $((P \vee Q) \wedge (\neg P \vee R)) \rightarrow (Q \vee R)$ (SAT)
- $(P \wedge Q) \rightarrow (P \wedge Q)$ (SAT)
- $P \wedge Q \rightarrow P$ (SAT)
- $P \rightarrow (P \vee Q)$ (SAT)
- $((P \wedge Q) \wedge P) \rightarrow Q$ (SAT)

Windows taskbar: Type here to search, 5:42 PM 5/24/2026